

Nav-HUD Team Report

ECE 441 Fall 2024

Instructor: Prof. Jafar Saniie

Due: 11/20/2024

Members:

Michael Schmidt

Olumosope Kayode

Daniil Skvyrskyi

Table of Contents:

- Abstract
- Team Member Information
- Introduction
- Project Description
- Hardware
- Software
- Related Work
- Results & Discussion
- User Manuals
- Challenges
- Work Contribution
- Conclusion
- Future Work
- References
- Appendix

Abstract:

The leading cause of vehicular accidents is driver inattention, often attributed to distractions from mobile devices, infotainment systems, or overcomplicated navigation displays. To address this critical issue, we propose *Nav-HUD*, an intelligent navigational display system designed to prioritize driver focus and road safety. By leveraging minimalistic visual cues, simple user interface, and contextual data integration, Nav-HUD provides real-time navigation assistance without overwhelming the driver.

Key features of Nav-HUD include a *Heads-Up Display (HUD)* to project essential route information onto the display and directions on a transparent display, ensuring that drivers keep their eyes on the road. The system employs a user-friendly design to view other information such as media being played, and can be connected to the user's phone.

Through rigorous simulation and real-world testing, Nav-HUD aims to set a new standard for navigation aids, directly addressing the root causes of driver inattention and contributing to a significant reduction in car accidents. This approach underscores the importance of technology in enhancing driver focus and establishing safer roads for everyone.

Team Member Information:

		
Olumosope Kayode 4th Year Electrical Engineer	Daniil Skvyrskyi 4th Year Computer Engineer	Michael Schmidt 4th Year Computer Engineer

Introduction:

Nav-HUD is a next-generation navigation display system designed to address the critical issue of driver inattention, the leading cause of vehicular accidents. By utilizing a combination of minimalist HUD (Hheads-Up Display) technology, projection of the route, and media display, Nav-HUD ensures that drivers remain focused on the road while receiving essential navigation information, regardless of the vehicles you own.

Project Description:

Description:

Nav-HUD aims to reduce the likelihood of getting distracted while driving, by creating our own navigation system that only displays essential information to the driver. Our user-friendly UI is designed such that it would not retain the attention of drivers and only display convenient information such as navigational directions or what media is currently playing.

The device works with the raspberry pi working as the central brain of the system. The GPS module and 12v battery will be plugged into the Pi for power and location data, then the Pi will do its calculations and display a navigation route on the touch screen. Finally the Pi will display a turn signal on the OLED display whenever there is a turn coming up.

Standards and Compliance

Nav-HUD adheres to the following standards to ensure compatibility, safety, and reliability:

- **Bluetooth (IEEE 802.15):** Used for wireless connection between the HUD and the driver's phone to stream GPS and media data.
- **I2C:** For communication between the microcontroller and display.
- **USB:** For power supply or wired connections if necessary.

System Constraints

- **Hardware Limitations:** The HUD system must be compact and lightweight to easy be portable
- **Power Consumption:** AttentiveNav must operate efficiently to minimize its impact on the vehicle's battery system, especially in electric vehicles. The display and microcontroller must operate efficiently, either using the car's 12V power supply or an external battery.
- **Real-Time Processing:** The system must process navigation data, driver monitoring inputs, and ADAS alerts in real time with minimal latency to ensure safety-critical actions are timely.
- **Scalability:** The system must be adaptable to various vehicle models, including those with different windshield sizes and configurations. The HUD must be compact enough to attach to the car's windshield without obstructing the driver's view.
- **Environmental Resilience:** The display system must function reliably in extreme temperatures, humidity, and lighting conditions, ensuring usability across diverse climates.
- **Cost:** The solution must be low-cost to remain affordable, especially for owners of older cars.
- **Connectivity:** Ensuring seamless wireless integration with various phone models via Bluetooth or Wi-Fi.
- **Security:** Ensuring secure data transmission between the phone and the HUD to prevent unauthorized access.

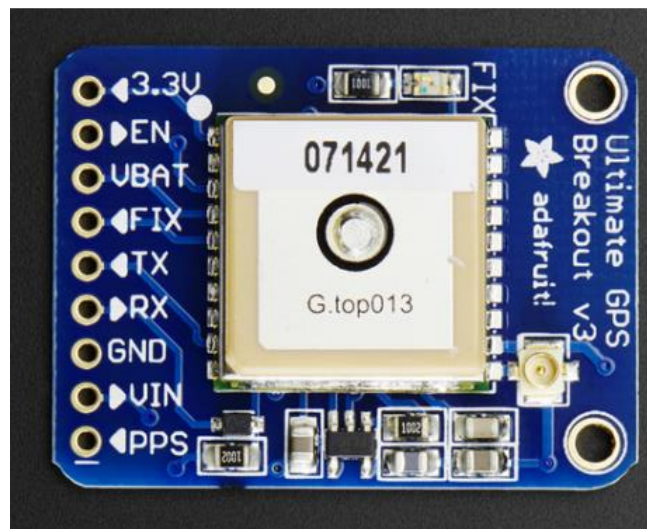
Safety Concerns

1. **Distraction Mitigation:** Overloading the driver with unnecessary information could negate the benefits of the system. Hav-HUD employs a strict UI system that only shows what are essential to the driver
2. **Interference with Visibility:** HUD projections are carefully calibrated to avoid obscuring the driver's view of the road. A display was chosen not to take up too much of the driver's view
- 3.
4. **Overheating issues:** Nav-HUD is carefully designed such that it can run fine in the ambient temperatures of the car. The Raspberry Pi used can run as normal under high heat of a car that has been out in the sun all day. There is currently no back case so the ambient air helps cool the device.

Hardware

This project is built around the Raspberry Pi 4, a powerful and versatile single-board computer developed by the Raspberry Pi Foundation. The Raspberry Pi 4 is equipped with a quad-core CPU, sufficient RAM for multitasking, and an extensive range of connectivity options, making it an excellent choice for various embedded systems and IoT applications. It supports multiple interfaces for peripherals, including USB ports, HDMI for video output, GPIO pins for custom hardware connections, and built-in networking options such as Ethernet, Wi-Fi, and Bluetooth. Additionally, it runs on a Linux-based operating system, allowing for high levels of reprogrammability and compatibility with a wide range of software. Its small size, lightweight design, and portability make it particularly well-suited for mobile and embedded applications, making it the perfect "brain" for the Nav-HUD device.

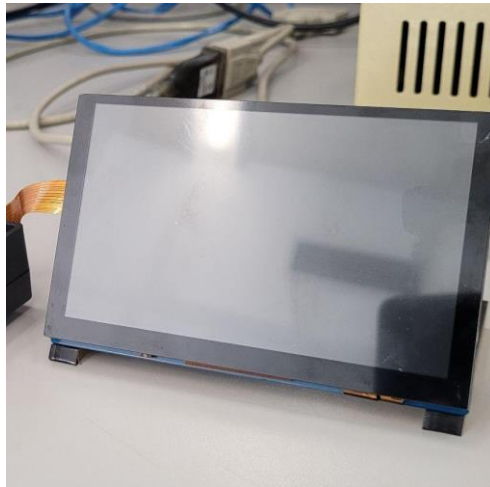
The Nav-HUD device relies on three key peripherals to deliver its functionality. The first is the Adafruit Ultimate GPS module, which provides highly accurate location data essential for navigation. This module ensures the system can track its position with precision and reliability, making it indispensable for the project. The second peripheral is the Raspberry Pi Touch Screen, a versatile display that serves as the user interface for the Nav-HUD. Its touch capabilities allow users to interact directly with the system, providing an intuitive and seamless experience. Finally, the Waveshare OLED module adds an additional layer of information display, offering clear and sharp visuals that enhance the usability of the device in different lighting conditions.



To enable the acquisition and transmission of positional and navigational data for processing by the Raspberry Pi, the Adafruit Ultimate GPS module was selected. This module is designed to operate seamlessly in a 5V system and requires only 20mA of current, making it an energy-

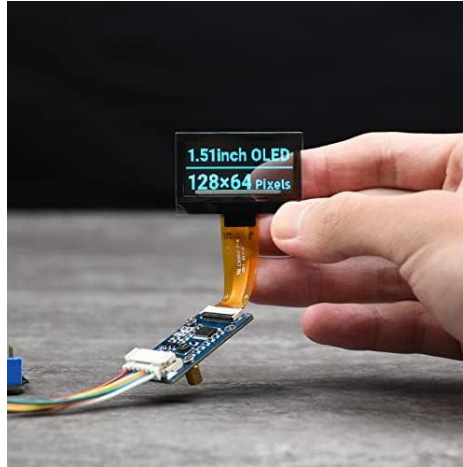
efficient choice. Its low power consumption is ideal, as it will be powered directly by the Raspberry Pi, which has a maximum current output of 1A from its 5V pin.

The Adafruit Ultimate GPS module provides impressive performance, capable of updating its location up to 10 times per second. It boasts high sensitivity for both logging and tracking, ensuring accurate and reliable location data even in challenging environments. Additionally, the module offers excellent position and velocity tracking, making it a robust solution for the Nav-HUD device's navigation requirements.



The Raspberry Pi Touch Screen was selected to serve as the primary display module for presenting processed information from the Raspberry Pi and functioning as the user interface (UX) device. This LCD display features a resolution of 800x480 pixels, providing clear and detailed visuals. It has a generous viewing area measuring 154.08 x 85.92 mm, making it suitable for displaying navigation data and user inputs effectively.

Additionally, the touch screen is designed to operate reliably across a wide temperature range, from -20°C to 70°C, ensuring durability and performance in various environmental conditions. This combination of functionality, clarity, and robustness makes it an excellent choice for the Nav-HUD device.



The Waveshare OLED module was chosen to provide clear and concise navigational instructions to the driver. One of its standout features is its transparent background, which ensures that it remains unobtrusive, reducing distractions while driving. This makes it particularly well-suited for applications like the Nav-HUD device, where maintaining the driver's focus is paramount. The OLED module has a resolution of 128x64 pixels, delivering sharp and clear visuals despite its compact size. The display area measures 35.05 x 15.32 mm, making it small enough to integrate seamlessly into the overall design while still providing sufficient space for essential navigation information.

The integration of the various modules with the Raspberry Pi is a key aspect of the system's design. The GPS module is connected using a Universal Asynchronous Receiver/Transmitter (UART) setup. This method allows for effective two-way communication between the GPS module and the Raspberry Pi through the RX (receive) and TX (transmit) pins. The UART setup ensures a reliable and efficient data exchange, enabling the Raspberry Pi to process precise location data received from the GPS module.

The touch screen connects to the Raspberry Pi via a Display Serial Interface (DSI) connector. This type of connection simplifies the design by enabling the DSI cable to carry both power and data from the Raspberry Pi to the display. This eliminates the need for additional power cables, contributing to a clean and organized hardware layout. The touch screen serves as the primary user interface, providing an interactive platform for displaying processed navigation data and allowing for user interaction.

The OLED module, responsible for delivering real-time navigation instructions to the driver, interfaces with the Raspberry Pi using a Serial Peripheral Interface (SPI) setup. The SPI connection ensures a fast and reliable communication link, crucial for updating instructions without delay. Power for the OLED module is supplied by the Raspberry Pi's 3.3V pin, eliminating the need for an external power source. This integration reduces complexity and enhances the device's portability.

To enclose and protect the hardware, the Raspberry Pi 4 case was selected as the housing for the Nav-HUD device. This case is specifically designed to securely hold the Raspberry Pi in place, providing a sturdy frame that ensures durability during operation. It also includes precise port cutouts, allowing easy access to all necessary connectors for peripherals like the GPS module, touch screen, and OLED module. The case's design not only offers physical protection for the components but also contributes to the device's aesthetic appeal and practicality.

Overall, the careful selection and integration of these components—combined with the robust and functional housing—ensure that the Nav-HUD device is both reliable and user-friendly, meeting the requirements for a portable, efficient, and distraction-free navigation system.

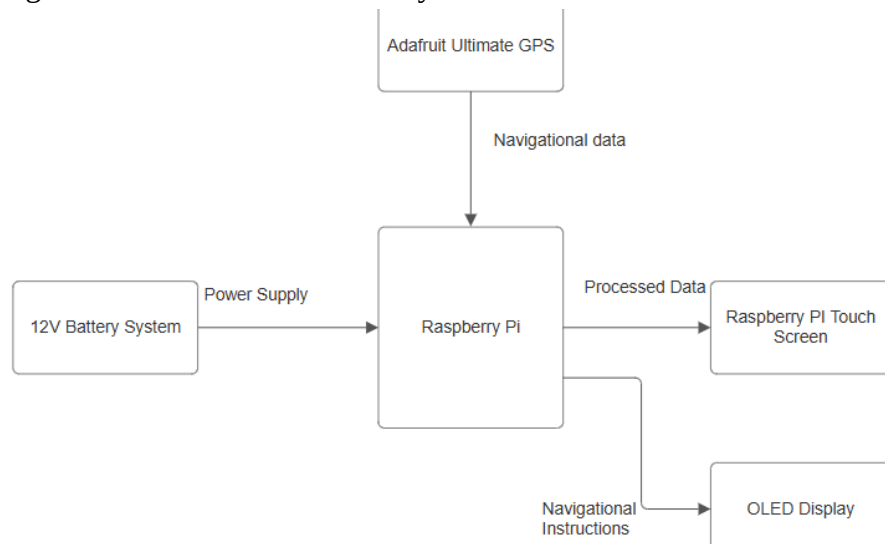


To ensure the device remains securely in place within a moving vehicle, double-sided adhesives were chosen as the fastening solution. These adhesives offer several advantages: they are simple to use, require no additional tools or complicated setup, and provide users the flexibility to position the Nav-HUD device wherever they prefer within their vehicle. This approach allows for both convenience and customization based on individual preferences and vehicle layouts.

For power, the device will rely on an external source: the car's 12V battery system. Since the Nav-HUD operates on a 5V power system, it will connect to the car's battery through a USB port or a cigarette lighter adapter. This setup allows for seamless integration with the vehicle's existing power infrastructure, ensuring consistent and reliable energy supply while maintaining compatibility with standard automotive power outlets.



A block diagram of the overall hardware system can be seen below:



In the diagram above, It can be seen that the Raspberry Pi will receive power from the 12V battery system. The Raspberry will then power and communicate with the Adafruit Ultimate GPS to receive navigational data. The Raspberry Pi will then transfer the processed data to the raspberry touch screen so the user can receive the desired information. The Pi will also send navigational instructions to the OLED display to display for the driver without having them take their eyes off the road.

Software

- **Functionality:** What functions are achieved using your software?
 - **Map generation:** Map generation code serves multiple functions, beginning with input handling to integrate geographical or spatial data from databases, APIs, or files, and to process user inputs such as location, scale, or map style. It may also handle real-time data updates, like traffic or weather. The code then creates map layers, starting with a base layer for foundational elements such as terrain or water bodies, and overlays feature layers for roads, buildings, or points of interest. It also allows for the addition of custom layers, enabling users to personalize the map with specific annotations or overlays.

Implementation Example:

Below is a snippet illustrating how the map is generated using the OSMNX library to plot the street network:

```
import osmnx as ox
```

```
import matplotlib.pyplot as plt
```

```
# Load the saved map graph
```

```
G = ox.load_graphml("chicago_drive_UPDATED.graphml")

def generate_default_map():
    # Create a default map image
    fig, ax = ox.plot_graph(
        G,
        show=False,
        close=True,
        bgcolor='black',
        node_size=0,
        edge_color='gray'
    )
    fig.savefig("default_map.png", dpi=100, bbox_inches='tight')
    plt.close(fig)

generate_default_map()
```

Explanation:

Loading the Graph:

```
G = ox.load_graphml("chicago_drive_UPDATED.graphml")
```

This line loads a pre-saved GraphML file containing the street network data for Chicago, which serves as the foundation for map generation and routing functionalities.

Plotting the Map:

```
fig, ax = ox.plot_graph(
    G,
    show=False,
    close=True,
    bgcolor='black',
    node_size=0,
    edge_color='gray'
)
```

The `plot_graph` function visualizes the street network with a black background and gray edges, creating a clear and uncluttered map image.

- **Estimate Time of Arrival Calculations:** Estimated Time of Arrival (ETA) calculations in code involve determining the time it will take to reach a destination based on various factors. The process typically begins with the calculation of the total distance along the planned route, which is derived from mapping data or pathfinding algorithms like Dijkstra's or A*. The average speed for each segment of the route is then considered, which may vary depending on

the type of road, speed limits, or real-time conditions such as traffic or weather. The code divides each segment's distance by its corresponding speed to calculate the time required for that segment, then sums these times to produce the total travel duration. Advanced implementations may incorporate live updates, adjusting the ETA dynamically as conditions change or as the route is altered. These calculations often account for mode-specific factors, such as vehicle acceleration, stops, or terrain, to enhance accuracy. The resulting ETA is presented in a user-friendly format, typically as a countdown or a clock-based arrival time.

Implementation Example:

Here's how ETA is calculated based on the route length and average speed:

```
from datetime import timedelta

def calculate_eta(route, avg_speed_kmh=50):
    route_length = 0
    for u, v in zip(route[:-1], route[1:]):
        # Retrieve the length of each segment
        length = G.get_edge_data(u, v)[0]['length'] # Assuming single edge
        route_length += length

    avg_speed_mps = avg_speed_kmh * 1000 / 3600 # Convert km/h to m/s
    travel_time_seconds = route_length / avg_speed_mps
    return timedelta(seconds=travel_time_seconds)
```

Explanation:

Route Length Calculation:

```
for u, v in zip(route[:-1], route[1:]):
    length = G.get_edge_data(u, v)[0]['length']
    route_length += length
```

This loop iterates through each pair of consecutive nodes in the route to sum up the total distance.

ETA Computation:

```
avg_speed_mps = avg_speed_kmh * 1000 / 3600
travel_time_seconds = route_length / avg_speed_mps
return timedelta(seconds=travel_time_seconds)
```

The average speed is converted from kilometers per hour to meters per second, and the total travel time is calculated by dividing the route length by this speed. The result is returned as a timedelta object for easy formatting.

- **Rendering and style:** Rendering and styling play a crucial role, converting spatial data into visual elements and applying rules for colors, shapes, and labels. The code dynamically adjusts detail based on zoom levels and manages geographic projections for accurate representation. For navigational maps, the code includes pathfinding and routing capabilities, calculating optimal routes between points, incorporating real-time updates like traffic, and estimating travel times. Labeling features dynamically position text for cities, roads, and landmarks, resolving conflicts to ensure readability, while offering user annotation capabilities for customization.

Interactivity enhances user engagement by enabling zooming, panning, and clickable elements that provide additional information. Search functionality allows users to locate places or features, and live updates keep the map current. The code is optimized for performance through techniques like data simplification, caching, and efficient resource usage, ensuring smooth rendering even with large datasets. Output options include generating static images or interactive maps, and in some cases, rendering 3D maps for visualizing elevation or urban landscapes.

- **Route generation:** Route generation code is responsible for determining the optimal path between a starting point and a destination based on predefined criteria such as shortest distance, fastest time, or user preferences. It uses algorithms like Dijkstra's or A* to traverse a graph representation of the map, where nodes represent locations and edges represent possible routes. Each edge is weighted according to factors like distance, travel time, or road conditions. The code processes this graph to find the most efficient path, often incorporating additional data such as real-time traffic updates, road closures, or construction zones to refine the route. Advanced implementations may also consider multi-modal routes, combining different modes of transport (e.g., walking and public transit). The generated route includes step-by-step directions and waypoints, which are formatted for display and navigation, ensuring clarity and usability. This process is dynamic, allowing for recalculations in real-time if the user deviates from the planned path or new conditions arise.

Implementation Example:

Below is a simplified example of how a route is planned using NetworkX's shortest path algorithm:

```
import networkx as nx
import osmnx as ox
from geopy.geocoders import Nominatim

def plan_route(G, start_lat, start_lon, dest_address):
```

```

# Geocode the destination address
geolocator = Nominatim(user_agent="st_hud")
location = geolocator.geocode(dest_address)
dest_lat, dest_lon = location.latitude, location.longitude

# Find the nearest nodes in the graph
orig_node = ox.nearest_nodes(G, X=start_lon, Y=start_lat)
dest_node = ox.nearest_nodes(G, X=dest_lon, Y=dest_lat)

# Calculate the shortest path
route = nx.shortest_path(G, orig_node, dest_node, weight='length')
return route

```

Explanation:

Geocoding Destination:

```
location = geolocator.geocode(dest_address)
dest_lat, dest_lon = location.latitude, location.longitude
```

Converts the destination address into geographic coordinates.

Identifying Nearest Nodes:

```
orig_node = ox.nearest_nodes(G, X=start_lon, Y=start_lat)
dest_node = ox.nearest_nodes(G, X=dest_lon, Y=dest_lat)
```

Finds the closest nodes in the graph to the starting and destination coordinates.

Calculating Shortest Path:

```
route = nx.shortest_path(G, orig_node, dest_node, weight='length')
```

Utilizes Dijkstra's algorithm to determine the shortest path based on the 'length' attribute of edges.

- **Route projection:** Route projection code is responsible for mapping a calculated route onto a visual representation of a map, ensuring it aligns accurately with the geographical context. This involves converting the route's geographic coordinates (latitude and longitude) into the map's projection system, such as Mercator or Lambert, so the path aligns correctly with the visual framework of the map. The code smooths the route by interpolating points and optimizing curves for readability, especially for complex or winding paths. It layers the route onto the map, often using distinct styling like colored lines or arrows to make it visually prominent. Advanced implementations may dynamically adjust the projection to ensure the entire route fits within the viewport or to maintain clarity as users zoom or pan. Additionally, the projection code may integrate real-time updates, modifying the route's visual representation

in response to changes like traffic or user deviations. This ensures the displayed route is accurate, visually clear, and contextually relevant for navigation.

- **Media retrieval:** Media retrieval code is designed to fetch multimedia content such as images, videos, or audio from various sources and prepare it for use within an application. This process often begins with accessing media files from local storage, cloud services, or remote servers through APIs or file systems. The code handles different file formats, ensuring compatibility by converting or transcoding media into a standardized format if necessary. In the case of remote media, it may utilize efficient transfer protocols like HTTP or streaming mechanisms to download or buffer content dynamically. Advanced implementations can handle authentication for secure sources, manage caching to reduce load times, and support fallback mechanisms in case of missing or inaccessible media. Metadata extraction is often included to retrieve information such as file size, resolution, duration, or tags, enabling further organization or processing of the content.
- **Media display:** The display component of the code focuses on rendering the retrieved media within the application's interface. It optimizes the content for the target device, considering factors like screen resolution, aspect ratio, and performance capabilities. For images, the code may handle scaling, cropping, or applying filters to ensure the visuals are clear and fit seamlessly within the UI. Video display involves managing playback controls, synchronization with audio tracks, and adapting quality dynamically based on bandwidth availability. Audio rendering ensures proper playback through compatible output devices while offering user controls for volume or playback speed. Interactive elements such as clickable hotspots or captions may also be layered over the media. In real-time or streaming scenarios, the code ensures smooth transitions between content, buffering management, and error handling to maintain a seamless user experience. Together, the retrieval and display components provide robust and efficient media management, enabling applications to deliver rich and engaging multimedia content.
- **Bluetooth connection:** Bluetooth connection code enables devices to establish and manage wireless communication using the Bluetooth protocol. The process typically begins with initializing the Bluetooth adapter on the host device to scan for nearby discoverable devices. Once detected, the code pairs the devices by exchanging authentication credentials, such as PINs or encryption keys, to ensure a secure connection. After pairing, the code establishes a communication channel using profiles like Serial Port Profile (SPP) for data transfer or Advanced Audio Distribution Profile (A2DP) for audio streaming. It manages tasks like establishing connections, maintaining the session, and handling potential interruptions or disconnections. Advanced implementations support bidirectional

data exchange, device role switching (e.g., master to slave), and multiple simultaneous connections. Error handling routines ensure stability by managing failed connections, timeouts, or signal interference. Additionally, the code may include options for custom configurations, such as setting device names or visibility preferences, to enhance usability.

Implementation Example:

Here is a snippet demonstrating how the Bluetooth media listener is set up using dbus-python and PyGObject:

```
import dbus
import dbus.mainloop.glib
from gi.repository import GLib
import threading

def ble_media_listener():
    # Initialize the D-Bus main loop
    dbus.mainloop.glib.DBusGMainLoop(set_as_default=True)
    bus = dbus.SystemBus()

    def properties_changed(interface, changed, invalidated, path=None):
        if interface != "org.bluez.MediaPlayer1":
            return
        # Handle media metadata changes here

    # Add a signal receiver for PropertiesChanged
    bus.add_signal_receiver(
        properties_changed,
        dbus_interface="org.freedesktop.DBus.Properties",
        signal_name="PropertiesChanged",
        path_keyword="path"
    )

    # Start the GLib main loop to listen for signals
    loop = GLib.MainLoop()
    loop.run()

# Start the listener in a separate thread
listener_thread = threading.Thread(target=ble_media_listener, daemon=True)
listener_thread.start()
```

Explanation:

Initializing D-Bus Loop:

```
dbus.mainloop.glib.DBusGMainLoop(set_as_default=True)
```

```
bus = dbus.SystemBus()
```

Sets up the D-Bus main loop to listen for Bluetooth media events.

Signal Receiver Setup:

```
bus.add_signal_receiver(  
    properties_changed,  
    dbus_interface="org.freedesktop.DBus.Properties",  
    signal_name="PropertiesChanged",  
    path_keyword="path"  
)
```

Registers a callback function `properties_changed` to handle incoming `PropertiesChanged` signals from Bluetooth media players.

Running the Listener:

```
loop = GLib.MainLoop()
```

```
loop.run()
```

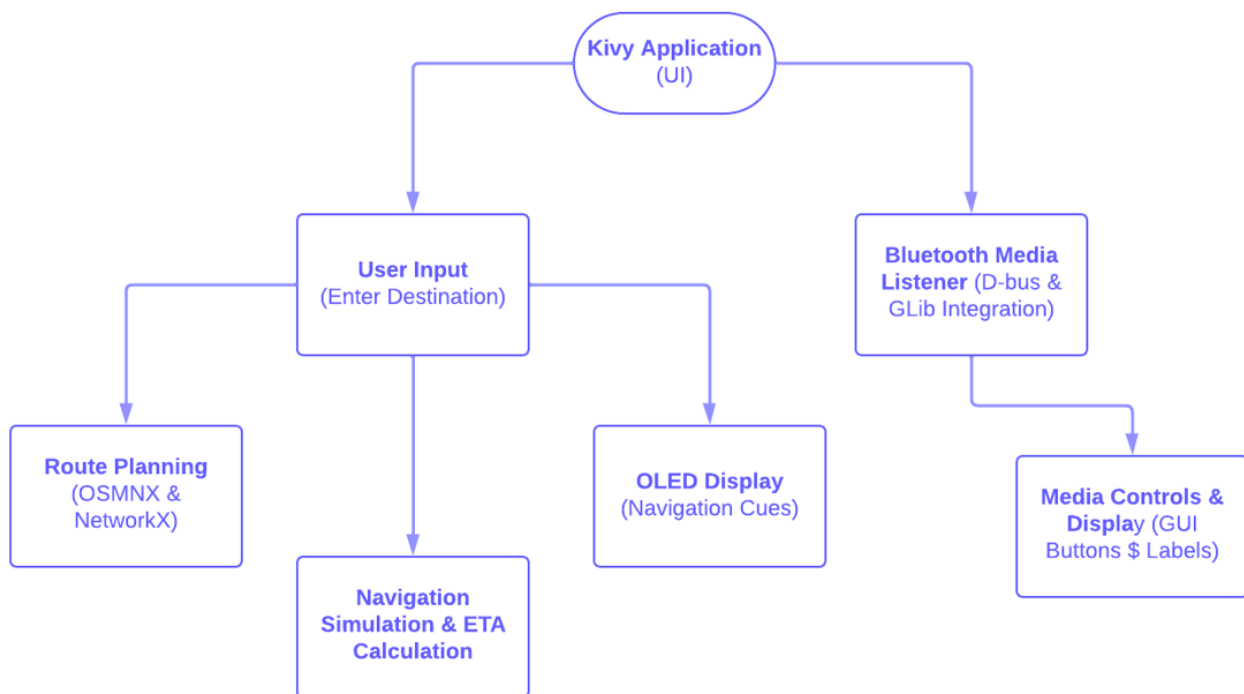
Starts the main loop in a separate thread to continuously listen for Bluetooth events without blocking the main application.

- **Programming language:** which one did you use? Why? Briefly explain the tradeoffs between the language you chose, and other possible languages
 - This project used Python to write the code in. Python is an excellent choice for writing navigational code due to its extensive library ecosystem, ease of use, and flexibility. Libraries like Geopy, OSMNx, and Pyproj simplify tasks such as geocoding, routing, and map projections, while its seamless integration with mapping APIs (e.g., Google Maps, OpenStreetMap) and real-time data handling makes it highly versatile. Python's intuitive syntax allows for rapid development and prototyping, and its robust support for machine learning through libraries like TensorFlow and scikit-learn enables predictive analytics and adaptive navigation features. Additionally, Python's cross-platform compatibility ensures it runs on a wide range of devices, from desktops to embedded systems like Raspberry Pi. Despite being slower than some compiled languages, its speed is often sufficient, and its performance can be enhanced with optimized libraries. With strong community support and portability, Python is an ideal choice for developing modern navigation systems.
- **Security:** Describe any security issues that might encounter on your system.

Navigational code raises security concerns, including data privacy risks from handling sensitive location information, API key exposure, and vulnerabilities to attacks

such as man-in-the-middle (MitM), denial of service (DoS), and GPS spoofing. Unsecured data transmission, unsanitized user inputs, and reliance on outdated dependencies increase the risk of breaches, unauthorized access, and system exploitation. Physical devices like Raspberry Pi also face risks of tampering or theft. To mitigate these issues, developers should use encryption (e.g., SSL/TLS), secure API keys, sanitize inputs, implement strong authentication, monitor data for anomalies, and keep libraries up to date. Addressing these concerns ensures the navigation system remains secure, reliable, and user-focused

- **Code:** Explain how much of the code is original to you or your group
 - Daniel: Bluetooth, Media retrieval, route projection/generation, map generation
 - Michael: Estimated Time of Arrival, UI design



Related Work:

Holometer by WOSLXM:

The Holometer is a holographic speedometer that projects speed data onto your windshield. It offers features such as speed display in both MPH and KM/H, overspeed alarms, fatigue warnings, and a GPS compass for direction. The device automatically adjusts its brightness based on ambient light conditions, ensuring optimal visibility during both day and night driving. It's designed for easy installation and is compatible with vehicles equipped with an OBD2 interface.



5.5" HUD Head Display by VEGBY:

The VEGBY HUD features a 5.5-inch display that projects vehicle speed, engine RPM, water temperature, voltage, and more onto the windshield. It includes overspeed alarms, fatigue driving reminders, and automatic brightness adjustment. The device is compatible with vehicles that have an OBDII or EUOBD interface and is designed for plug-and-play installation.



Auto HUD Display Car Projector Alarm by ZJCGO:

The ZJCGO HUD is tailored for specific vehicle models, such as the Toyota RAV4 XA50 (2019–2024). It projects speed and other vital information onto the windshield, helping drivers maintain focus on the road. The device includes features like overspeed alarms and is designed to integrate seamlessly with the vehicle's existing systems.

2JC60

For Nissan X-Trail Rogue XTrail

T32 2013 – 2022

Head Up Display

HUD



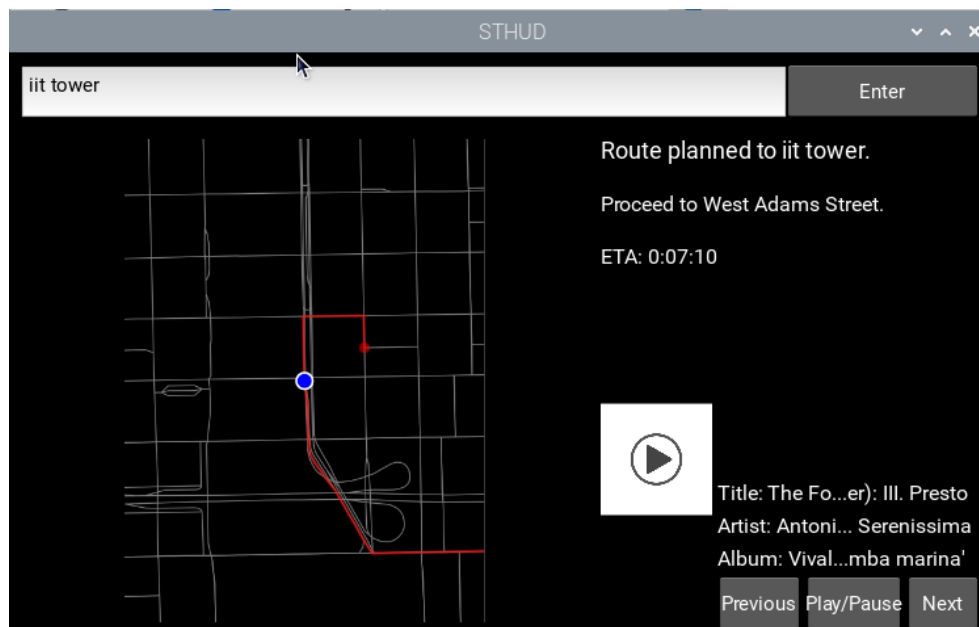
Car Head Display GPS Windshield Projector by Apexeon:

The Apexeon HUD utilizes GPS to project speed and navigation information onto the windshield. It's suitable for vehicles without an OBDII interface, as it doesn't rely on the car's onboard diagnostics system. The device offers features like speed display, overspeed alarms, and automatic brightness adjustment to ensure clear visibility under various lighting conditions.



Results & Discussion

Device works as intended. Has a display for the route, bluetooth connectivity, displays media being played. Upon launching the system, users are asked to input a destination. With a keyboard they can type in their desired location. After a few seconds the map will change and start projecting the route users must take. Users can also connect to the device with their phones via bluetooth to display audio that they are playing via their phone. They can also control the audio from the device by skipping the song, going back, and pausing and playing it.



User's Manual of our system

1. Begin by powering the device
2. Enter the destination in the search bar field
3. Wait a seconds for the device to do calculations
4. The map should change and show the route to take
5. Follow the route until destination is met
6. (Optional) Connect to the device via bluetooth
7. (Optional) Device will now display media that is currently being played

Challenges:

- GPS configuration: We've encountered errors with our gps not reading any data to give us our positions
- Map Generation: There was difficulty researching how we can generate the map data, and give real-time navigational instructions on the map
- Bluetooth Connectivity: There were difficulties setting up the bluetooth connection to pull media information from user's phones to display the data
- Dynamic Zoom: There were issues with the way we setup the code to give a zoomed in view of the surrounding areas of the user while there were moving

Team Member Work Contributions

- **Daniil Skvytskyi:**
 - Code for Map Generation
 - Code for Bluetooth Connectivity
 - Code for Media retrieval
 - Code for Route Calculations
- **Olumosope Kayode:**
 - Hardware Selection
 - Project Design
 - Schematics Design
- **Michael Schmidt:**
 - Hardware Selection
 - Code for ETA Module

- Code for UI Design
- Code for Zoom Functionality

Conclusion

As of the end of this course, the hardware was fully assembled and functional. The GPS module and touch screen were fully connected to the Raspberry Pi 4 and were functioning as intended. A successful bluetooth connection was established between the device and the user's phone. The bluetooth connection allowed the user to make use of media controls and control the spotify platform through the Raspberry Pi. Accurate route planning and ETA calculations were also implemented in the device. The results of the planning and calculations were also displayed in the UI. The device is also self-sufficient; it does not need a connection to WI-FI or the user's phone to perform the navigational tasks. Although the device is not capable of using its media functions without the phone connection, this does not raise concern because that is a non-essential feature.

Future Work

- Find Larger display that is transparent to improve on distracted driving
- Create a better interface to interact with the device, currently you need a keyboard to type. Possibility of voice recognition
- Add options to change or select songs
- Add Auto-Fatigue Detection module to alert drivers when fatigued
- Implement system to collect real-time data on traffic and accidents coming up

References

- Raspberry Touch Screen:
 - <https://datasheets.raspberrypi.com/display/7-inch-display-product-brief.pdf>
 - <https://www.raspberrypi.com/documentation/accessories/display.html>
- Raspberry PI:
 - <https://datasheets.raspberrypi.com/rpi4/raspberry-pi-4-datasheet.pdf>
- Adafruit Ultimate GPS:
 - <https://cdn-learn.adafruit.com/downloads/pdf/adafruit-ultimate-gps.pdf>
- Dual Car Charger, USB Type C:
 - <https://www.assmann-wsw.com/uploads/datasheets/84104.pdf>
- Python documentation:
 - <https://docs.python.org/3/installing/index.html>
- Waveshare 1.51" Transparent Display:
 - <https://www.waveshare.com/1.51inch-transparent-oled.htm>

- [https://www.waveshare.com/wiki/1.51inch Transparent OLED](https://www.waveshare.com/wiki/1.51inch_Transparent_OLED)

Part List

Part Name / number	Amount required	Price
Double Sided Tape Heavy Duty	1	\$9.99
3M VHB Double-Sided Sticker Adhesive Pad	1	\$12.90
Raspberry Pi Touch Display	1	\$60.00
Adafruit Ultimate GPS	1	\$29.95
Raspberry Pi 4	1	\$55.00
Raspberry Pi 4 Case	1	\$11.99
waveshare 1.51inch Transparent OLED 128×64	1	\$14.50

Appendix

py:

```
import osmnx as ox
import os
import networkx as nx
from geopy.geocoders import Nominatim
import matplotlib.pyplot as plt
import matplotlib
matplotlib.use('Agg') # Use Agg backend for non-GUI environments

from kivy.app import App
from kivy.uix.floatlayout import FloatLayout
from kivy.lang import Builder
from kivy.properties import StringProperty, ListProperty
```

```

from kivy.clock import mainthread
from kivy.config import Config
from datetime import timedelta

Config.set('graphics', 'width', '600')
Config.set('graphics', 'height', '380')

# Load the kv file explicitly
Builder.load_file("st_hud.kv")

# Load the saved map graph
G = ox.load_graphml("chicago_drive_UPDATED.graphml")

class STHUDLayout(FloatLayout):
    bluetooth_status = StringProperty("Not Connected")
    address = StringProperty("Enter Address")
    route_info = StringProperty("Route will be displayed here")
    navigation_instruction = StringProperty("Navigation instructions will
appear here")
    map_image_path = StringProperty("default_map.png") # Initial map
image

    #Edited code for ETa- Michael
    #
    #
    #
    eta_info = StringProperty("ETA will be displayed here")
    #End Edits

    def __init__(self, **kwargs):
        super(STHUDLayout, self).__init__(**kwargs)
        # Generate an initial map image if needed
        self.G = G # Assign the global graph to an instance variable
        if not os.path.exists(self.map_image_path):
            self.generate_default_map()

    def generate_default_map(self):
        # Create a default map image
        fig, ax = ox.plot_graph(G, show=False, close=True,
bgcolor='black', node_size=0, edge_color='gray')
        fig.savefig(self.map_image_path, dpi=100, bbox_inches='tight')
        plt.close(fig)

    @mainthread
    def on_address_entered(self, address_text):
        self.address = address_text.strip()

```

```

    if not self.address:
        self.route_info = "Please enter a valid address."
        return
    self.route_info = f"Planning route to: {self.address}"
    # Start route planning
    self.plan_route_to_address(self.address)

#Edited code for ETA - Michael
#
#
#
#
#Calculate ETA
def calculate_eta(self, route, avg_speed_kmh=50):
    route_length = 0
    for u, v in zip(route[:-1], route[1:]):
        # Check each multi-edge between nodes u and v
        length = None
        for key, data in self.G.get_edge_data(u, v).items():
            length = data.get('length')
            if length:
                route_length += length
                break # Stop after finding the first valid length for
multi-edges
        if length is None:
            print(f"Warning: Missing length for edge ({u}, {v}). Using
default length of 50 meters.")
            route_length += 50 # Default length in meters if missing

    avg_speed_mps = avg_speed_kmh * 1000 / 3600
    travel_time_seconds = route_length / avg_speed_mps
    return timedelta(seconds=travel_time_seconds)

#End Edits

def plan_route_to_address(self, dest_address):
    # Use predefined location or GPS data
    use_gps = False # Set to True to use GPS data
    if not use_gps:
        start_lat, start_lon = 41.878876, -87.635915 # Willis Tower
    else:
        start_lat, start_lon = self.get_current_location()
        if start_lat is None or start_lon is None:
            self.route_info = "GPS data not available."
            return

```

```

# Plan the route
route = self.plan_route(G, start_lat, start_lon, dest_address)
if route:
    self.route_info = f"Route planned to {dest_address}."
    # Plot the route and update the map image
    self.plot_route(G, route)
    # Calculate and display ETA
    eta = self.calculate_eta(route)
    self.eta_info = f"ETA: {str(eta)}"
    # Generate turn-by-turn instructions
    instructions = self.generate_turn_by_turn_instructions(G,
route)

    self.navigation_instruction = instructions
else:
    self.route_info = f"Failed to plan the route to
{dest_address}."
    self.navigation_instruction = "No navigation instructions
available."
    # Optionally, reset the map image to default
    self.map_image_path = "default_map.png"

def get_coordinates_from_address(self, address):
    geolocator = Nominatim(user_agent="st_hud")
    try:
        location = geolocator.geocode(address)
        if location:
            return location.latitude, location.longitude
        else:
            print(f"Address '{address}' not found.")
            return None, None
    except Exception as e:
        print(f"Geocoding error: {e}")
        return None, None

def plan_route(self, G, start_lat, start_lon, dest_address):
    # Get destination coordinates from address
    dest_lat, dest_lon =
self.get_coordinates_from_address(dest_address)
    if dest_lat is None or dest_lon is None:
        print("Invalid destination address.")
        return None

    # Get nearest nodes to the start and destination coordinates
    try:
        orig_node = ox.nearest_nodes(G, X=start_lon, Y=start_lat)
        dest_node = ox.nearest_nodes(G, X=dest_lon, Y=dest_lat)
    except Exception as e:

```

```

        print(f"Error finding nearest nodes: {e}")
        return None

    # Calculate the shortest path
    try:
        route = nx.shortest_path(G, orig_node, dest_node,
weight='length')
        return route
    except Exception as e:
        print(f"Error calculating shortest path: {e}")
        return None

def plot_route(self, G, route):
    import time
    timestamp = int(time.time())
    image_path = f"route_map_{timestamp}.png"

    # Get bounding box around the route for dynamic zoom
    lats = [G.nodes[node]['y'] for node in route]
    lons = [G.nodes[node]['x'] for node in route]
    north, south = max(lats), min(lats)
    east, west = max(lons), min(lons)
    margin = 0.01 # Add margin to the bounding box

    fig, ax = ox.plot_graph_route(
        G,
        route,
        show=False,
        close=True,
        bgcolor='black',
        node_size=0,
        edge_color='gray',
        route_color='red',
        route_linewidth=3,
        bbox=(north + margin, south - margin, east + margin, west -
margin)
    )
    fig.savefig(image_path, dpi=100, bbox_inches='tight')
    plt.close(fig)
    self.map_image_path = image_path # Update the image path with new
filename

def generate_turn_by_turn_instructions(self, G, route):
    # Simple placeholder implementation
    instructions = "Proceed along the highlighted route."
    # More advanced instruction generation can be implemented here

```

```

        return instructions

def get_current_location(self):
    # Implement GPS data retrieval
    try:
        import gpssd
        gpssd.connect()
        packet = gpssd.get_current()
        if packet.mode >= 2: # Mode 2D or 3D fix
            return packet.lat, packet.lon
        else:
            print("GPS has no fix.")
            return None, None
    except Exception as e:
        print(f"GPS error: {e}")
        return None, None

class STHUDApp(App):
    def build(self):
        # Ensure default_map.png exists before building the UI
        layout = STHUDLayout()
        return layout

if __name__ == "__main__":
    STHUDApp().run()

```

kv:

<STHUDLayout>:

BoxLayout:

orientation: 'vertical'

padding: dp(10)

spacing: dp(10)

Button:

id: bluetooth_button

text: "Connect via Bluetooth"

size_hint_y: None

height: dp(40)

Remove or comment out the on_press action

on_press: root.bluetooth_status = "Connecting..."

Label:

id: bluetooth_status_label
text: root.bluetooth_status
size_hint_y: None
height: dp(20)
font_size: '18sp'

BoxLayout:

orientation: 'horizontal'
size_hint_y: None
height: dp(40)

TextInput:

id: address_input
text: root.address
multiline: False
size_hint_x: 0.8
on_text_validate: root.on_address_entered(self.text)

Button:

text: "Enter"
size_hint_x: 0.2
on_press: root.on_address_entered(address_input.text)

Image:

id: map_image
source: root.map_image_path
allow_stretch: True
keep_ratio: True
size_hint_y: None
height: dp(200)

Label:

id: route_info_label
text: root.route_info
font_size: '18sp'
size_hint_y: None
height: dp(20)

Label:

id: navigation_instruction_label

text: root.navigation_instruction
font_size: '16sp'
size_hint_y: None
height: dp(40)

Label:

text: root.eta_info
pos_hint: {"x": 0.1, "y": 0.7}
size_hint: 0.8, 0.1
font_size: 16
color: 1, 1, 1, 1 # White color